Documentazione Tecnica

Sistema di Controllo per Drone ad Ala Fissa

*Analisi architetturale del firmware embedded sviluppato su piattaforma **Teensy** per la gestione autonoma di un velivolo a pilotaggio remoto con capacità di navigazione GPS e controllo di volo mediante algoritmi PID.*

Piattaforma:	Teensy (ARM Cortex-M)
Linguaggio:	C++ / Arduino Framework
Versione FW:	1.0
Classificazione:	Uso Tecnico Interno

Indice

1 Panoramica Architetturale **4**

2 GPS e Interfacce di Comunicazione **4**
2.1 Modulo GPS 4
2.2 Ricevente S.BUS — FrSky 4
2.3 Telemetria LoRa 5
2.4 GPS UART 5

3 Procedura di Inizializzazione (`setup()`) **5**
3.1 Fase 1 — Inizializzazione Hardware di Base 5
3.2 Fase 2 — Ciclo di Inizializzazione Sensoriale 5
3.2.1 IMU BNO055 6
3.2.2 Barometro BMP390 6
3.2.3 Tubo di Pitot 7
3.2.4 Moduli INA219 7
3.3 Fase 3 — Gestione degli Errori Critici 8
3.4 Fase 4 — Inizializzazione dei Servomeccanismi 9
3.5 Fase 5 — Avvio del Timer PID 9

4 Gestione Ridondata dell'Alimentazione e Logica di Failover **9**

5 Rilevamento delle Collisioni e Gestione delle Emergenze Cinematiche **10**

6 Acquisizione Sensoriale Integrata e Navigazione Autonoma **12**
6.0.1 Telemetria Attuatori (Health Monitoring) 12
6.1 Cinematica e Profilo Altimetrico 12
6.2 Anemometria, Cinematica GPS e Sensor Fusion 13
6.3 Algoritmo di Navigazione Non-Lineare (L1 Guidance Law) 14

7 Preparazione Adattiva del Profilo Propulsivo **14**
7.1 Logica di Selezione della Fase di Volo 14
7.2 Fase di Crociera 15
7.3 Fase di Avvicinamento Finale 15

8 Lettura del Ricevente S.BUS e Selezione della Modalità **15**
8.1 Acquisizione del Frame S.BUS 16
8.2 Decodifica del Canale di Selezione Modalità 16
8.3 Arbitraggio Finale: Failsafe come Override Prioritario 16

9 Macchina a Stati della Modalità di Volo **17**

10 Reset dei Controllori PID al Cambio di Modalità **17**
10.1 Rilevamento della Transizione di Stato 17
10.2 Procedura di Reset delle Variabili di Stato PID 17
10.3 Sblocco di Emergenza Post-Schianto 18

11 Modalità Manuale — Mappatura Diretta dei Canali RC **19**
11.1 Condizione di Attivazione 19
11.2 Mappatura dei Canali RC sugli Attuatori 19
11.2.1 Gas (Canale 3 — `canaliRC[2]`) 19
11.2.2 Rollio (Canale 1 — `canaliRC[0]`) 19
11.2.3 Beccheggio (Canale 2 — `canaliRC[1]`) 20
11.3 Ottimizzazione dell'Output Seriale 20

12 Modalità Automatica e Failsafe — Controllo PID **20**
12.1 Condizione di Attivazione 20
12.2 Controllo di Volo Autonomo: Architettura PID a Cascata 21
12.2.1 Gestione del Dominio Temporale (*Time-Step Protection*) 21
12.2.2 Parametri Operativi del Controllore 21
12.2.3 Loop 1: Altitudine e *Pitch-Targeting* 22
12.2.4 Loop 2 e 3: Dinamica d'Assetto (Pitch e Roll) 22
12.2.5 Loop 4: Autothrottle e Gestione dell'Energia Cinetica 23

13 Attuazione Fisica, Override di Sicurezza e Watchdog Hardware **23**
13.1 Logica di Attuazione e Inibizione Motore (Crash Override) 23
13.2 Gestione Dinamica dell'Interfaccia Visiva (HMI) 24

14 Allocazione dei Comandi e Matrice di *Mixing* Adattiva **25**
14.1 Risparmio Energetico Attivo (*Load Shedding*) 25
14.2 Matrice Cinematica e Riconfigurazione Strutturale 25
14.3 Gestione Dinamica dell'Hardware (Attach/Detach) 26
14.4 Saturazione Meccanica e Attuazione Finale 26

15 Watchdog Seriale del Sottosistema GNSS **27**
15.1 Sistema di Telemetria 27
15.1.1 Acquisizione Dati 27
15.1.2 Codifica degli Allarmi 28
15.1.3 Struttura del Pacchetto 28
15.1.4 Esempio di Pacchetto Telemetrico 29
15.1.5 Formato e Robustezza 30

15.1.6 Considerazioni Sistemistiche 30

1. Panoramica Architetturale

Il firmware è concepito come un sistema di controllo di volo (*Flight Control System*, FCS) completamente embedded, destinato a un velivolo ad ala fissa con configurazione *flying wing* a quattro superfici mobili. Il codice integra in un unico ciclo di esecuzione la lettura di una rete di sensori eterogenea, l'elaborazione di algoritmi di controllo in retroazione (*closed-loop*), la navigazione autonoma verso coordinate GPS predefinite e la trasmissione di dati telemetrici in tempo reale tramite modem LoRa.

Il microcontrollore adottato — un **Teensy** basato su nucleo ARM Cortex-M — garantisce la potenza computazionale necessaria per la gestione concorrente di più bus seriali e I^2C, indispensabile date la molteplicità dei sensori installati a bordo.

Nota Tecnica

L'intera architettura segue il paradigma *sense–compute–actuate*: i sensori acquisiscono lo stato fisico del velivolo, il firmware elabora i dati attraverso i controllori PID, e gli attuatori (servomotori e motore brushless) imprimono le correzioni necessarie alle superfici di controllo.

2. GPS e Interfacce di Comunicazione

2.1. Modulo GPS

Il modulo GPS è collegato alla porta seriale hardware `Serial1` alla velocità di `9600` baud. Il parser *TinyGPS++* elabora il flusso di sentenze NMEA (tipicamente `$GPGGA` e `$GPRMC`), estraendo posizione geografica, velocità sul suolo e numero di satelliti agganciati. La velocità GPS viene successivamente fusa con la stima basata sul tubo di Pitot mediante un filtro passa-basso di primo ordine.

Un sensore di temperatura analogico NTC/PTC è collegato al pin `A1` (`PIN_T_motore`). Il valore campionato viene memorizzato nella variabile globale `Global_Temperatura_motore` e concorre alla diagnostica termica del gruppo propulsore, prevenendo condizioni di surriscaldamento.

2.2. Ricevente S.BUS — FrSky

Il protocollo **S.BUS** è uno standard seriale unidirezionale operante a `100 kbaud`, `8E2` (8 bit dati, parità pari, 2 bit di stop), con segnale logicamente invertito. La libreria `SBUS` gestisce trasparentemente la decodifica del frame, restituendo i valori di 16 canali RC

nell'array `canaliRC[16]`, nonché i flag `failsafe` e `pacchettoPerso` per la rilevazione di perdita del segnale di controllo.

2.3. Telemetria LoRa

Il downlink telemetrico si avvale della porta seriale `Serial4` (`TELEMETRIA`), operante a `9600 baud`, connessa a un modem LoRa (*Long Range*) che consente trasmissioni a lunga distanza con bassa potenza. I pacchetti telemetrici, inviati periodicamente tramite la funzione `inviaTelemetria()`, veicolano lo stato completo del velivolo: assetto, velocità, posizione GPS, uscite dei PID e stato dei servomeccanismi.

2.4. GPS UART

La comunicazione con il modulo GPS avviene sulla porta `Serial1` alla velocità standard di `9600 baud`. Il firmware legge il flusso NMEA in modalità non bloccante all'interno del loop principale, aggiornando l'oggetto `gps` tramite il metodo `encode()`.

3. Procedura di Inizializzazione (`setup()`)

La funzione `setup()` costituisce la sequenza di avvio critica del firmware. La sua esecuzione è strutturata in fasi sequenziali e bloccanti, concepite per garantire che il sistema raggiunga l'operatività piena prima di cedere il controllo al loop principale.

3.1. Fase 1 — Inizializzazione Hardware di Base

Vengono configurati:

- La porta seriale di debug (`Serial`, 115200 baud)
- Il bus I^2C primario (`Wire`) a 400 kHz (*Fast Mode*)
- I pin di uscita per LED, buzzer e relè
- Lo stato iniziale basso (inattivo) di tutte le uscite di segnalazione

Al termine di questa fase, il sistema emette una **sequenza sonora di accensione a tre toni ascendenti** (800 Hz → 1200 Hz → 1600 Hz), fungendo da conferma uditiva dell'avvio corretto dell'hardware di base.

3.2. Fase 2 — Ciclo di Inizializzazione Sensoriale

Il sistema tenta l'inizializzazione di tutti i moduli sensoriali in un ciclo `while` che si ripete fino a un massimo di **`MAX_TENTATIVI = 3` iterazioni**, o fino alla riuscita completa. In ogni iterazione vengono verificati:

3.2.1. IMU BNO055

Il **BNO055** è un dispositivo *System-in-Package* che integra accelerometro, giroscopio e magnetometro triassiali, elaborandone internamente la fusione sensoriale tramite un algoritmo AHRS (*Attitude and Heading Reference System*) proprietario. Ciò consente al firmware di ricevere direttamente angoli di orientamento assoluti (rollio, beccheggio, imbardata) senza la necessità di implementare filtri di stima dedicati (Kalman, Mahony) a livello applicativo.Viene invocato il metodo `giroscopio.begin()` che tenta la comunicazione I^2C con il sensore. In caso di successo, viene attivato l'oscillatore cristallino esterno (`setExtCrystalUse(true)`) per massimizzare la precisione del timing interno. Il flag `imuPronto` registra l'esito e si accende il LED verde e il buzzer (tono acuto a 1200Hz) per 300ms; in caso di errore, segnala con 3 lampeggi del LED rosso e 3 beep brevi del buzzer.

3.2.2. Barometro BMP390

Successivamente viene inizializzato il sensore barometrico **BMP390**, un dispositivo ad alta precisione per la misura di pressione atmosferica e temperatura. L'inizializzazione avviene tramite interfaccia I^2C e prevede la verifica della corretta comunicazione con il dispositivo; in caso positivo, il sensore viene configurato impostando opportunamente i registri interni per ottimizzare il compromesso tra risoluzione, rumore e frequenza di aggiornamento.

In particolare, vengono applicate le seguenti configurazioni operative:

- Oversampling della temperatura a 8× e della pressione a 32×, al fine di ridurre il rumore di quantizzazione e migliorare la risoluzione effettiva delle misure
- Filtro IIR con coefficiente pari a 3, utilizzato per attenuare le componenti ad alta frequenza del segnale di pressione (ad esempio dovute a vibrazioni o turbolenze)
- Output Data Rate impostato a 50 Hz, per garantire un adeguato aggiornamento temporale delle misure

Una volta configurato, il sensore **BMP390** fornisce misure di pressione atmosferica e temperatura con risoluzione sub-aPa, consentendo la stima dell'altitudine mediante la formula ipsometrica:

$$h = \frac{T_0}{L}\left[\left(\frac{p}{p_0}\right)^{-\frac{RL}{gM}} - 1\right]$$

dove $p_0 = 1013.25\,\text{hPa}$ rappresenta la pressione di riferimento al livello del mare secondo il modello ISA.

Successivamente, viene eseguita una procedura di taratura dell'altitudine iniziale: vengono acquisite 20 letture consecutive di quota, con lampeggio del **LED blu** a

cadenza regolare come feedback visivo dello stato della calibrazione. I valori acquisiti vengono mediati per determinare la quota di riferimento h_0 (memorizzata nella variabile `global_altitudineDiPartenza`). Tale valore viene sottratto alle misure successive per ottenere l'**altitudine dal suolo** (*Above Ground Level*, AGL).Se l'operazione va a buon fine, accende il LED verde e il buzzer (tono acuto a 1200Hz) per 300ms; in caso di errore, segnala con 3 lampeggi del LED rosso e 3 beep brevi del buzzer.

3.2.3. Tubo di Pitot

Il sensore di pressione differenziale associato al tubo di Pitot è connesso al pin analogico `A0`. La tensione differenziale tra la presa di ristagno e la presa statica viene convertita in pressione dinamica e, successivamente, in velocità aerodinamica mediante la relazione di Bernoulli:

$$v = \sqrt{\frac{2 \cdot \Delta p}{\rho}}$$

dove $\rho = 1.225\,\mathrm{kg/m^3}$ rappresenta la densità dell'aria in condizioni ISA standard. Un fattore di conversione empirico (`FATTORE_CONVERSIONE_PA = 3.22`) consente di tradurre il valore ADC grezzo nella corrispondente pressione differenziale effettiva.

Durante la fase di inizializzazione, il sensore viene sottoposto a una procedura di azzeramento (*tara*) al fine di determinare il valore di offset (`VALORE_ZERO`). Tale procedura prevede l'acquisizione di 100 campioni ADC con il velivolo fermo e in condizioni di aria calma, successivamente mediati per ottenere una stima robusta del valore di riferimento:

$$\texttt{VALORE_ZERO} = \frac{1}{100} \sum_{i=1}^{100} \texttt{ADC}_i$$

Durante questa fase, il sistema fornisce un feedback visivo attraverso il **LED verde** (associato al GPS), che lampeggia a intervalli regolari (ogni 10 campioni acquisiti) per indicare che la calibrazione è in corso. Al termine della procedura, il LED viene spento e il buzzer disattivato, segnalando la conclusione del processo.

La validità della calibrazione viene verificata controllando che il valore medio ottenuto rientri nell'intervallo $(5, 1020)$. Valori al di fuori di tale range sono considerati anomali e indicativi di possibili malfunzionamenti, quali cortocircuiti o disconnessioni del sensore.Se l'operazione va a buon fine, accende il LED verde e il buzzer (tono acuto a 1200Hz) per 300ms; in caso di errore, segnala con 3 lampeggi del LED rosso e 3 beep brevi del buzzer.

3.2.4. Moduli INA219

Successivamente, il sistema di bordo del drone dispone di un sottosistema dedicato al monitoraggio energetico in tempo reale, basato su sei moduli **INA219**. Tali dispositivi

consentono la misura simultanea di tensione e corrente sui rami di alimentazione critici, permettendo una valutazione continua dello stato energetico e dell'assorbimento dei vari sottosistemi.

Ciascun modulo è configurato con un indirizzo I^2C distinto (o separato tramite bus fisicamente indipendenti) mediante resistori di indirizzamento:

Variabile	**Indirizzo I^2C**	**Ramo monitorato**
`sensoreMotore`	0x40 (Wire)	Batteria principale / ESC motore
`sensoreTeensy`	0x40 (Wire1)	Alimentazione logica Teensy
`sensoreIntSX`	0x41	Servo flap interno sinistro
`sensoreIntDX`	0x42	Servo flap interno destro
`sensoreEstSX`	0x43	Servo flap esterno sinistro
`sensoreEstDX`	0x43 (Wire1)	Servo flap esterno destro

Nota Tecnica

L'utilizzo del bus I^2C secondario `Wire1` per i sensori `sensoreEstDX` e `sensoreTeensy` consente di aggirare i conflitti di indirizzo tra dispositivi INA219 configurati con lo stesso indirizzo (`0x43` e `0x40`), distribuendo il traffico su due bus fisicamente separati e migliorando l'affidabilità della comunicazione.

Durante la fase di inizializzazione, tutti e sei i moduli INA219 vengono avviati sequenzialmente tramite interfaccia I^2C. Per ciascun dispositivo viene verificata la corretta comunicazione; un eventuale fallimento nell'inizializzazione di uno o più sensori comporta l'impostazione del flag globale `Voltaggio` a `false`, causando la ripetizione del processo fino al corretto riconoscimento di tutti i moduli.

3.3. Fase 3 — Gestione degli Errori Critici

Qualora al termine dei tre tentativi uno o più tra IMU, barometro e Pitot non risultassero operativi, il sistema entra in uno stato di **blocco permanente** (*hard fault*): il **LED rosso** rimane acceso fisso e il buzzer emette un tono continuo a 2000 Hz, segnalando la necessità di intervento fisico sull'hardware prima del riavvio.

Avvertenza

In condizioni di errore critico, il firmware non cede mai il controllo al `loop()`: il velivolo rimane a terra con i servomeccanismi non inizializzati, impedendo qualsiasi tentativo di volo in stato degradato.

Tabella 1: Segnalazione LED e Buzzer per inizializzazione, calibrazione e allarmi

Situazione	**LED**	**Comportamento**	**Buzzer**
Successo inizializzazione sensore	Verde GPS	ON 300 ms	1200 Hz, 150 ms
Errore sensore	Rosso Alarm	3 lampeggi	400 Hz, 100 ms
Calibrazione barometro	Blu PID	Lampeggio alternato	Buzzer alternato
Calibrazione Pitot	Verde GPS	Lampeggio alternato	Buzzer alternato
Tentativo fallito (non critico)	Rosso Alarm	ON 2 s	-
Errore critico (blocco drone)	Rosso Alarm	ON fisso	2000 Hz ciclico

3.4. Fase 4 — Inizializzazione dei Servomeccanismi

Con tutti i sensori operativi, i quattro servomeccanismi e l'ESC vengono attaccati ai rispettivi pin PWM e posizionati nelle condizioni iniziali di sicurezza:

- Superfici di controllo: posizione neutra (`CENTRO_SERVO = 90°`)
- Motore: gas minimo (`GAS_MINIMO = 25°`)

La procedura si conclude con un *jingle* sonoro tritonale ascendente e un lampo simultaneo di tutti e tre i LED, a conferma visiva e acustica del sistema completamente operativo.

3.5. Fase 5 — Avvio del Timer PID

L'istruzione `tempoPassatoPID = millis()` registra il timestamp di inizio, inizializzando il riferimento temporale per il calcolo dei termini integrale e derivativo dei controllori PID nel loop principale.

4. Gestione Ridondata dell'Alimentazione e Logica di Failover

All'interno del ciclo di esecuzione principale dell'aeromobile, il firmware richiama ciclicamente la routine dedicata alla supervisione energetica. Questa sezione del sistema operativo in tempo reale è di importanza critica, in quanto garantisce la tolleranza ai guasti (*Fault Tolerance*) del sottosistema di alimentazione avionica e previene il catastrofico arresto dei sistemi di volo.

Come prima operazione, l'algoritmo effettua il *polling* dei sensori INA219 sul bus I^2C, estraendo in particolare la tensione della linea logica (`vTeensy`) e la tensione del pacco batterie di propulsione (`vMotore`).

Successivamente, viene valutato lo stato della batteria di servizio dedicata alla logica. Il valore `vTeensy` viene confrontato con una soglia critica fissata a 4.9 V. Qualora tale

soglia venga violata, il sistema attiva un meccanismo di ridondanza energetica basato su una commutazione hardware.

In particolare, il pin GPIO 20 viene portato a livello logico `HIGH`, pilotando un relè elettromeccanico posto tra i due circuiti di alimentazione (batteria di servizio e linea BEC proveniente dall'ESC). Il relè è configurato in modo tale che, in condizioni nominali, la logica di volo sia alimentata esclusivamente dalla batteria dedicata.

Quando si verifica la condizione di sottotensione:

- il relè commuta collegando la linea di alimentazione della Teensy al BEC del motore;
- un diodo di isolamento (in serie al percorso di backup) consente il flusso di corrente dalla batteria di trazione verso la logica, impedendo al contempo correnti inverse verso la batteria esausta.

In questo modo, quando la batteria di servizio scende sotto la soglia critica, l'alimentazione del microcontrollore viene automaticamente commutata sulla linea di backup proveniente dal BEC del motore.

La batteria principale di trazione interviene quindi come sorgente secondaria, sostenendo il sistema avionico e garantendo la continuità operativa della logica di volo, della navigazione e della telemetria anche in condizioni di guasto della sorgente primaria.

L'evento di commutazione aggiorna le variabili di stato globali, che vengono trasmesse alla Ground Control Station (GCS) per fornire consapevolezza situazionale all'operatore.

Infine, la routine esegue un'analisi parallela sul pacco batterie primario (LiPo 4S). La tensione `vMotore` viene confrontata con una soglia di sicurezza pari a 11.8 V. Un eventuale superamento negativo di tale limite attiva un *flag* globale, successivamente codificato nella *bitmask* telemetrica per generare allarmi visivi e acustici lato operatore, indicando la necessità di un atterraggio immediato.

5. Rilevamento delle Collisioni e Gestione delle Emergenze Cinematiche

Il firmware implementa un modulo di sicurezza attiva dedicato al monitoraggio continuo dell'integrità strutturale dell'aeromobile e alla mitigazione dei danni post-impatto. Tale routine, richiamata ad ogni iterazione del ciclo di controllo principale, analizza le variazioni cinematiche per discriminare le normali manovre di volo da eventi di decelerazione anomala assimilabili a uno schianto (*Crash Detection*).

L'architettura logica della funzione si basa su un modello a macchina a stati finiti. Come prima istruzione, il sistema verifica lo stato della variabile booleana di blocco globale (`statoSchiantoRilevato`). Qualora l'aeromobile si trovi già in stato di emergenza conclamata, la funzione elude qualsiasi ulteriore calcolo e inietta forzatamente il comando di *Idle* (`GAS_MINIMO`) sul bus PWM diretto all'Electronic Speed Controller (ESC). Questa prassi di *latching* (ritenzione dello stato) garantisce che nessuna interferenza successiva all'impatto possa riarmare accidentalmente la propulsione.

Per prevenire l'innesco di falsi positivi (*Spurious Triggers*) durante la manipolazione a terra, il trasporto a mano o l'armamento in pista, l'algoritmo subordina l'attivazione del rilevamento a un filtro di *Flight Envelope Validation*. Il monitoraggio degli urti viene infatti "armato" solo ed esclusivamente se l'aeromobile soddisfa almeno una delle due condizioni telemetriche di volo confermato: una **velocità** aerodinamica stimata superiore a 5.0 m/s o un'**altitudine** relativa dal suolo maggiore di 5.0 m. Finché la variabile globale **`droneInVolo`** non viene validata da queste metriche, la routine viene bypassata, rendendo l'Unmanned Aerial Vehicle (UAV) inerte agli urti da movimentazione.

Una volta validato lo stato di volo, il sistema entra nella fase di acquisizione cinematica ad alta frequenza. Tramite il protocollo I2C, il microcontrollore interroga il Digital Signal Processor (DSP) integrato nell'Inertial Measurement Unit (BNO055), richiedendo l'estrazione del vettore di accelerazione lineare. Tale vettore è epurato, per calcolo hardware interno al sensore, dal vettore della gravità terrestre ($1g$), restituendo esclusivamente le accelerazioni nette impresse al telaio lungo gli assi X, Y e Z.

Successivamente, il processore esegue il calcolo della norma euclidea (magnitudine) del vettore tridimensionale, ricavando la forza totale di accelerazione o decelerazione istantanea impressa al baricentro del drone, secondo l'equazione:

$$a_{tot} = \sqrt{a_x^2 + a_y^2 + a_z^2} \tag{1}$$

Il valore scalare ottenuto, espresso in m/s^2, viene comparato in tempo reale con la soglia critica di tolleranza pre-impostata a 25 m/s^2 (pari a circa $2.5g$ netti). Una sollecitazione che superi tale limite meccanico è ingegneristicamente incompatibile con le normali dinamiche del volo ad ala fissa, indicando in modo inequivocabile un urto balistico contro il terreno o un ostacolo.

La validazione positiva del superamento di soglia innesca istantaneamente il protocollo di *Catastrophic Failure*. Le azioni intraprese in frazioni di secondo prevedono:

- Il **taglio** definitivo dell'erogazione di potenza al motore (`GAS_MINIMO`), misura cruciale per evitare il surriscaldamento per stallo del propulsore, la combustione dell'ESC o lesioni causate dall'elica ancora in rotazione nell'erba o tra i detriti.

- La generazione di un **log seriale** di diagnostica estrema, con la stampa a terminale della forza esatta dell'impatto registrata al momento dell'interruzione.
- L'attivazione simultanea e permanente della triade di indicatori **LED** (*Rosso, Verde, Blu*) per fornire una risposta visiva immediata dello stato di avaria critica.
- L'innesco di un **segnale acustico** continuo ad alta frequenza (2000 Hz) tramite il trasduttore piezoelettrico di bordo. Tale emissione acustica funge da *Search and Rescue Beacon* (faro di soccorso), facilitando enormemente il recupero fisico dell'aeromobile in zone con fitta vegetazione o scarsa visibilità.

6. Acquisizione Sensoriale Integrata e Navigazione Autonoma

Il segmento centrale del ciclo di esecuzione (`loop`) costituisce il nucleo operativo dell'algoritmo di volo. In questa fase, il microcontrollore acquisisce i dati ambientali e cinematici, diagnostica lo stato degli attuatori e calcola la traiettoria ottima per il raggiungimento dei *waypoint* programmati.

6.0.1. Telemetria Attuatori (Health Monitoring)

La funzione `diagnosticaServi()` esegue un'analisi in tempo reale dell'assorbimento elettrico delle quattro superfici di controllo. Interrogando i sensori INA219, il sistema verifica che la corrente I di ogni singolo servomotore rientri in una stretta finestra operativa di sicurezza:

$$1 \text{ mA} < I < 2500 \text{ mA} \tag{2}$$

Il limite inferiore (1 mA) rileva istantaneamente un'avaria di tipo *Open Circuit* (es. cablaggio tranciato o connettore scollegato). Il limite superiore (2500 mA) previene danni a cascata intercettando condizioni di *Short Circuit* o stallo meccanico dell'attuatore (es. cerniera bloccata o sovraccarico aerodinamico). Se una condizione non è verificata, il relativo flag booleano di salute (es. `estSX_Ok`) viene invalidato, consentendo al *Mixer* a valle di isolare il guasto.

6.1. Cinematica e Profilo Altimetrico

L'assetto tridimensionale dell'aeromobile viene campionato prelevando le componenti vettoriali degli angoli di Eulero (Pitch, Roll, Yaw) processate internamente dal DSP del sensore inerziale BNO055. Parallelamente, il modulo barometrico fornisce la misurazione della pressione atmosferica assoluta, dalla quale viene derivata l'altitudine relativa al suolo (*Above Ground Level*, AGL) sottraendo la tara calcolata durante il *Pre-Flight Check*. Il codice acquisisce inoltre i parametri termici dell'avionica e del

propulsore, applicando per quest'ultimo la funzione di trasferimento lineare tipica dei trasduttori a semiconduttore (es. TMP36): $T(°C) = (V_{out} - 0.5) \cdot 100$.

6.2. Anemometria, Cinematica GPS e Sensor Fusion

La determinazione del vettore velocità dell'aeromobile rappresenta un parametro di importanza vitale sia per il sostentamento aerodinamico (prevenzione dello stallo) sia per la navigazione tattica. A tale scopo, il firmware implementa un'architettura di *Sensor Fusion* che accoppia la fluidodinamica classica alla radionavigazione satellitare.

La stima della velocità rispetto all'aria (*True Airspeed*, TAS) è affidata a un tubo di Pitot connesso a un trasduttore di pressione differenziale analogico. Il convertitore ADC a 10 bit del microcontrollore campiona il livello di tensione, dal quale viene sottratto l'offset di zero (`VALORE_ZERO`) pre-calibrato al suolo. Il dato netto viene vincolato a valori non negativi tramite una funzione di *clamping* (`constrain`) per garantire la stabilità nel dominio reale delle successive radici quadrate.

La differenza analogica viene scalata in Pascal (ΔP) e utilizzata per ricavare la velocità aerodinamica istantanea V_{pitot}, invertendo l'equazione di Bernoulli per i fluidi incomprimibili:

$$V_{pitot} = \sqrt{\frac{2\Delta P}{\rho}} \tag{3}$$

dove ρ denota la densità dell'aria in condizioni atmosferiche standard (1.225 kg/m^3).

Parallelamente all'anemometria, il modulo `TinyGPSPlus` estrapola la velocità cinematica rispetto al suolo (*Ground Speed*, GS). Il dato nativo, acquisito tramite la porta UART dedicata (`Serial1`) e originariamente espresso in chilometri orari, viene normalizzato nel Sistema Internazionale dividendo lo scalare per il fattore di conversione 3.6, ottenendo così la variabile V_{gps} in metri al secondo (m/s). In questa medesima istanza logica, il sistema provvede all'inizializzazione delle coordinate di riferimento geodetico al momento dell'acquisizione del primo *Fix* satellitare valido.

Poiché la sola misurazione anemometrica è soggetta a rumore ad alta frequenza e potenziali ostruzioni meccaniche, e la misurazione GPS è intrinsecamente affetta da latenza e incapace di rilevare le componenti di vento relativo (vento in prua o in coda), il firmware fonde i due vettori tramite un filtro complementare del primo ordine a guadagno fisso. La velocità stimata finale ($V_{stimata}$), che alimenterà i loop di controllo PID e le logiche di navigazione L1, è governata dall'equazione alle differenze:

$$V_{stimata} = \alpha \cdot V_{pitot} + (1 - \alpha) \cdot V_{gps} \tag{4}$$

Il coefficiente di smorzamento α (definito nel codice come `alpha_vel = 0.7`) assegna il 70% del peso algoritmico alla reattività istantanea del tubo di Pitot, parametro im-

prescindibile per la reiezione dello stallo, mentre il restante 30% del peso fa affidamento sulla robustezza a bassa frequenza del dato GPS, fungendo da correttore continuo per compensare eventuali derive (*drift*) del sensore di pressione differenziale.

6.3. Algoritmo di Navigazione Non-Lineare (L1 Guidance Law)

La funzione `aggiornaNavigazione()` rappresenta l'apice dell'autonomia tattica del sistema. Essa sfrutta la libreria `TinyGPSPlus` per calcolare la distanza geodetica (formula dell'Haversine) e la rotta ortodromica verso il target (`TARGET_LAT`, `TARGET_LON`).

Per guidare l'aeromobile sulla traiettoria desiderata evitando sovraelongazioni (*overshoot*), viene implementata la logica di guida non-lineare **L1**. L'algoritmo definisce innanzitutto una distanza di *look-ahead* L_1, proporzionale alla velocità istantanea dell'aeromobile moltiplicata per una costante di smorzamento (nel codice: 4.0 secondi). La velocità stimata viene preventivamente vincolata a un minimo di 1.0 m/s per evitare singolarità matematiche e divisioni per zero.

Sia η l'errore di rotta normalizzato nell'intervallo $[-\pi, \pi]$ (differenza tra la rotta verso il target e lo Yaw attuale della bussola). L'accelerazione laterale a_{lat} richiesta per curvare verso il punto L1 è definita dall'equazione geometrica:

$$a_{lat} = 2\frac{V^2}{L_1}\sin(\eta) \tag{5}$$

Tale accelerazione centripeta viene generata inclinando l'ala fissa. Il controllore calcola l'angolo di rollio bersaglio (ϕ_{target}) bilanciando la portanza inclinata con l'accelerazione di gravità g:

$$\phi_{target} = \arctan\left(\frac{a_{lat}}{g}\right) \tag{6}$$

L'angolo risultante viene convertito in gradi e sottoposto a *clamping* (`constrain`) in base al limite meccanico di rollio consentito (`MAX_ROLL`), fornendo un setpoint stabile e aerodinamicamente sicuro ai loop del controllore PID a valle.

7. Preparazione Adattiva del Profilo Propulsivo

7.1. Logica di Selezione della Fase di Volo

Prima di qualsiasi interazione con la radio o i controllori PID, il firmware determina il **regime propulsivo obiettivo** confrontando la distanza attuale dal waypoint di destinazione — calcolata al ciclo precedente dalla funzione `aggiornaNavigazione()`

e memorizzata nella variabile globale `global_distanzaDalTarget` — con la soglia di commutazione `DISTANZA_FRENATA`.

La discriminante è la seguente:

$$\text{Fase} = \begin{cases} \textbf{Crociera} & \text{se } d_{\text{target}} > 150\,\text{m} \\ \textbf{Avvicinamento} & \text{se } d_{\text{target}} \leq 150\,\text{m} \end{cases}$$

7.2. Fase di Crociera

Quando la distanza dal waypoint supera i **150 m** (`DISTANZA_FRENATA`), il sistema si trova in regime di crociera: il controllore di velocità riceve come riferimento `targetVelocita` = 60.0 km/h (`VELOCITA_CROCIERA`), mentre il valore di gas di base trasmesso al PID propulsivo è fissato a `gasDiBase` = 70ř (`GAS_CROCIERA`). Quest'ultimo rappresenta la posizione angolare nominale dell'ESC in regime stazionario, attorno alla quale il PID di velocità opera le proprie correzioni incrementali.

7.3. Fase di Avvicinamento Finale

All'ingresso nella **bolla di avvicinamento** (distanza dal target $\leq 150\,\text{m}$), il firmware riduce automaticamente i riferimenti propulsivi: la velocità obiettivo scende a **45 km/h** (`VELOCITA_AVVICINAMENTO`) e il gas di base viene abbassato a **45°** di posizione ESC (valore fisso, non associato a una costante nominata). La decelerazione ha lo scopo di contenere l'energia cinetica residua del velivolo nella fase terminale della missione, agevolando una traiettoria di atterraggio o di sgancio payload più controllata.

Fase	Distanza soglia	Velocità target	Gas di base (ESC)
Crociera	$> 150\,\text{m}$	60.0 km/h	70ř
Avvicinamento	$\leq 150\,\text{m}$	45.0 km/h	45ř

Nota Tecnica

I valori `targetVelocita` e `gasDiBase` non agiscono direttamente sull'ESC: fungono da *setpoint* e *punto di lavoro* per il controllore PID di velocità, che li elabora nel ciclo successivo generando il comando gas finale `comandoGasFinale`. La separazione tra riferimento e attuazione è intenzionale e garantisce la continuità del controllo in retroazione.

8. Lettura del Ricevente S.BUS e Selezione della Modalità

8.1. Acquisizione del Frame S.BUS

Il ricevente FrSky trasmette in modo continuo pacchetti S.BUS sul bus seriale `Serial3`. La libreria `SBUS` decodifica questi frame e ne espone il contenuto tramite il metodo `read()`, che popola:

- L'array `canaliRC[16]`: valori grezzi dei 16 canali RC nell'intervallo tipico $[172, 1811]$, corrispondenti rispettivamente allo stick al minimo e allo stick al massimo.
- Il flag booleano `failsafe`: attivato dal ricevente stesso qualora non riceva segnale valido dal trasmettitore per un periodo superiore alla soglia configurata.
- Il flag `pacchettoPerso`: segnala la perdita di singoli frame S.BUS, distinguibile dalla condizione di failsafe prolungato.

La chiamata a `read()` restituisce `true` solo in presenza di un nuovo frame valido e completamente decodificato, pertanto l'aggiornamento della modalità avviene esclusivamente in corrispondenza di dati freschi, evitando decisioni basate su campioni obsoleti.

8.2. Decodifica del Canale di Selezione Modalità

Il **canale 5** del radiocomando (indice `canaliRC[4]` nell'array zero-based) è dedicato alla selezione della modalità di volo. Si tratta tipicamente di un interruttore a due o tre posizioni sul trasmettitore, il cui valore grezzo viene confrontato con la soglia di **992 unità S.BUS**:

$$\texttt{global_modalitaVolo} = \begin{cases} 1\ (\textit{Manuale}) & \text{se } \texttt{canaliRC[4]} < 992 \\ 2\ (\textit{Auto GPS}) & \text{se } \texttt{canaliRC[4]} \geq 992 \end{cases}$$

Il valore 992 corrisponde alla posizione centrale del range S.BUS $[172, 1811]$, fungendo da soglia di bisezione netta tra i due stati dell'interruttore.

8.3. Arbitraggio Finale: Failsafe come Override Prioritario

Indipendentemente dalla selezione manuale del pilota, il **failsafe sovrascrive qualsiasi modalità** con priorità assoluta. La variabile `statoAttuale` — distinta dalla `global_modalitaVolo` per preservare quest'ultima intatta per logiche successive — viene impostata a **3** in presenza di failsafe, a prescindere dal valore letto sul canale 5.

$$\texttt{statoAttuale} = \begin{cases} 3 & \text{se } \texttt{failsafe} = \texttt{true} \\ \texttt{global_modalitaVolo} & \text{altrimenti} \end{cases}$$

Avvertenza

La separazione logica tra `global_modalitaVolo` e `statoAttuale` è una scelta architetturale deliberata: in condizione di failsafe, il sistema deve comportarsi come in modalità Auto (ricorrendo ai PID per il rientro autonomo), ma la variabile di modalità non deve essere alterata, affinché al ripristino del segnale radio il sistema torni allo stato selezionato dal pilota senza richiedere un'azione esplicita.

9. Macchina a Stati della Modalità di Volo

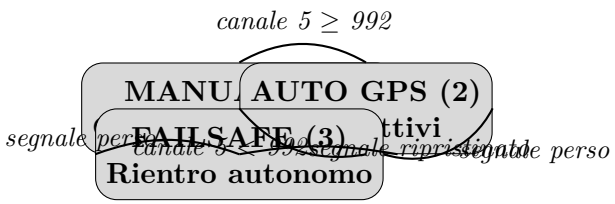

Figura 1: Diagramma di transizione della macchina a stati della modalità di volo. Le frecce in rosso indicano transizioni forzate dal failsafe, indipendenti dalla volontà del pilota.

10. Reset dei Controllori PID al Cambio di Modalità

10.1. Rilevamento della Transizione di Stato

Il firmware mantiene una variabile statica locale `modalitaPrecedente` — inizializzata a 1 (Manuale) — che viene confrontata ad ogni ciclo con `statoAttuale`. Una **discrepanza tra i due valori** segnala una transizione di modalità appena avvenuta, innescando la sequenza di reset.

L'utilizzo della qualifica `static` per questa variabile è essenziale: garantisce la persistenza del valore tra cicli successivi del loop, comportandosi di fatto come una variabile globale con scope locale alla funzione.

10.2. Procedura di Reset delle Variabili di Stato PID

In occasione di ogni transizione di modalità, **tutte le variabili di stato** dei quattro controllori PID vengono azzerate simultaneamente:

Variabile azzerata	Tipo	Controllore
`pid_sommaErroriAlt`	Integrale	Altitudine
`pid_errorePassatoAlt`	Derivativo	Altitudine
`pid_sommaErroriPitch`	Integrale	Beccheggio
`pid_errorePassatoPitch`	Derivativo	Beccheggio
`pid_sommaErroriRoll`	Integrale	Rollio
`pid_errorePassatoRoll`	Derivativo	Rollio
`pid_sommaErroriVel`	Integrale	Velocità
`pid_errorePassatoVel`	Derivativo	Velocità

Contestualmente, il riferimento temporale `tempoPassatoPID` viene aggiornato con il timestamp corrente tramite `millis()`, azzerando la storia temporale del controllore e impedendo che un Δt anomalo — conseguente alla pausa tra le modalità — generi un picco derivativo (*derivative kick*) al primo ciclo successivo alla transizione.

Nota Tecnica

L'azzeramento degli accumulatori integrali alla transizione di modalità è una pratica fondamentale nella progettazione di sistemi di controllo ibridi. In assenza di tale reset, la sommatoria accumulata nella modalità precedente — potenzialmente relativa a un assetto completamente diverso — verrebbe iniettata come disturbo all'ingresso degli attuatori, causando manovre violente e non intenzionali nei primi istanti della nuova modalità.

10.3. Sblocco di Emergenza Post-Schianto

La transizione di modalità veicola anche una funzione secondaria di fondamentale importanza operativa: il **ripristino del sistema dopo un evento di schianto rilevato**. Qualora il flag `statoSchiantoRilevato` risulti attivo al momento della transizione, il firmware interpreta il cambio di modalità operato dal pilota via radio come un **segnale di sblocco intenzionale** e procede con:

1. Azzeramento del flag `statoSchiantoRilevato`, riportando il sistema in uno stato di operatività normale.
2. Azzeramento del flag `droneInVolo`, resettando la logica di rilevamento del volo attivo.
3. Silenziamento del buzzer di allarme (`noTone()`), che durante lo schianto emette un segnale continuo di soccorso.
4. Emissione di una sequenza sonora di conferma a due toni (1000 Hz poi 1500 Hz, 100 ms ciascuno), che attesta acusticamente al pilota l'avvenuto sblocco e la disponibilità al riarmo.

5. Stampa di un messaggio diagnostico sulla porta seriale: *«SBLOCCO EMERGENZA ESEGUITO DA RADIO — Pronto al riarmo»*.

Avvertenza

Questo meccanismo di sblocco è deliberatamente vincolato a un'azione *esplicita* del pilota sul radiocomando: non è automatico né temporizzato. Ciò garantisce che il sistema non riprenda autonomamente l'operatività dopo un impatto senza la supervisione diretta dell'operatore in campo.

11. Modalità Manuale — Mappatura Diretta dei Canali RC

11.1. Condizione di Attivazione

La modalità manuale è attiva quando `global_modalitaVolo` è uguale a 1 *e* il flag `failsafe` è `false`. La congiunzione di entrambe le condizioni è imprescindibile: anche con il canale 5 in posizione manuale, il failsafe ha comunque la precedenza, escludendo questa modalità e delegando il controllo ai PID.

11.2. Mappatura dei Canali RC sugli Attuatori

In modalità manuale, i valori grezzi S.BUS dei tre canali primari di pilotaggio vengono convertiti nei rispettivi comandi agli attuatori mediante la funzione `map()` di Arduino, seguita da un `constrain()` che recide eventuali valori fuori range:

11.2.1. Gas (Canale 3 — `canaliRC[2]`)

Il canale del gas viene mappato linearmente dall'intervallo grezzo S.BUS $[172, 1811]$ all'intervallo meccanico dell'ESC $[25\check{r}, 130\check{r}]$:

$$\texttt{comandoGasFinale} = \text{clip}\left(\frac{(\texttt{canaliRC[2]} - 172)}{(1811 - 172)} \cdot (130 - 25) + 25,\ 25,\ 130\right)$$

dove i valori **25°** e **130°** corrispondono rispettivamente a `GAS_MINIMO` e `GAS_MASSIMO`, i limiti fisici di corsa dell'ESC configurati come costanti di sistema.

11.2.2. Rollio (Canale 1 — `canaliRC[0]`)

Il canale di rollio viene mappato sull'intervallo simmetrico $[-35\check{r}, +35\check{r}]$ (`MAX_ROLL`), con lo zero che corrisponde allo stick centrato (posizione neutra delle superfici):

$$\texttt{correzioneRoll} = \text{clip}\left(\frac{(\texttt{canaliRC[0]} - 172)}{(1811 - 172)} \cdot 70 - 35,\ -35,\ 35\right)$$

11.2.3. Beccheggio (Canale 2 — `canaliRC[1]`)

Il canale di beccheggio viene mappato sull'intervallo $[-20ř, +20ř]$ (`MAX_PITCH`). Si noti che la mappatura è **invertita** rispetto al gas e al rollio: il valore minimo S.BUS (172) corrisponde a `MAX_PITCH` e il valore massimo S.BUS (1811) corrisponde a `-MAX_PITCH`. Questa inversione riflette la convenzione aerodinamica standard del radiocomando in modalità *Mode 2*, per cui lo stick del beccheggio tirato verso il basso (*pull*) corrisponde a un angolo positivo (muso su):

$$\texttt{correzionePitch} = \text{clip}\left(\frac{(\texttt{canaliRC[1]} - 172)}{(1811 - 172)} \cdot (-40) + 20,\ -20,\ 20\right)$$

Canale	Indice	Range S.BUS	Output	Range output
Gas	`[2]`	$[172, 1811]$	`comandoGasFinale`	$[25ř, 130ř]$
Rollio	`[0]`	$[172, 1811]$	`correzioneRoll`	$[-35ř, +35ř]$
Beccheggio	`[1]`	$[172, 1811]$	`correzionePitch`	$[+20ř, -20ř]$ (inv.)

11.3. Ottimizzazione dell'Output Seriale

Per evitare la saturazione della porta seriale di debug con messaggi ridondanti — fenomeno particolarmente penalizzante in sistemi real-time dove la UART introduce latenza — il messaggio di stato *«Volo: MANUALE»* viene stampato **una sola volta per ogni transizione** verso la modalità manuale, confrontando `statoAttuale` con la variabile statica `ultimoStatoStampato`. La stampa avviene esclusivamente quando i due valori divergono, dopodiché `ultimoStatoStampato` viene aggiornato per sopprimere le stampe successive.

12. Modalità Automatica e Failsafe — Controllo PID

12.1. Condizione di Attivazione

Questa modalità è attiva in due scenari distinti, trattati dal firmware in modo funzionalmente identico seppur semanticamente differenziato:

- **Auto GPS (modalità 2)**: il pilota ha selezionato esplicitamente il controllo automatico tramite l'interruttore sul trasmettitore (canale $5 \geq 992$). Il drone naviga autonomamente verso il waypoint con i PID attivi.

- **Failsafe (modalità 3)**: il collegamento radio è stato perso. Il sistema attiva automaticamente i PID per portare il velivolo verso le coordinate target (`TARGET_LAT`, `TARGET_LON`) con la quota di sicurezza di **40 m AGL** (`ALTITUDINE_TARGET`), presumibilmente corrispondente al punto di decollo o a una zona sicura predefinita.

Nota Tecnica

La scelta di trattare failsafe e Auto GPS con la stessa logica di controllo non è una semplificazione, ma una precisa scelta di progetto: in entrambi i casi il velivolo deve raggiungere un punto geografico noto in modo autonomo, e i controllori PID sono lo strumento adeguato per farlo.

12.2. Controllo di Volo Autonomo: Architettura PID a Cascata

Il cuore pulsante della stabilizzazione del velivolo e della navigazione autonoma è demandato alla funzione `calcolaPID()`. Questa routine implementa un'architettura di controllo proporzionale-integrale-derivativo (PID) di tipo Multi-Input Multi-Output (MIMO). Per disaccoppiare la dinamica longitudinale da quella laterale, il sistema utilizza quattro anelli di retroazione (*feedback loops*) calcolati in sequenza ad ogni iterazione del loop principale.

12.2.1. Gestione del Dominio Temporale (Time-Step Protection)

Per garantire che le componenti integrali e derivative dell'equazione PID rimangano matematicamente coerenti, la funzione calcola il tempo trascorso (`dt`) dall'ultima iterazione, espresso in secondi. Per prevenire divergenze dell'algoritmo (fenomeno del *Windup*), il firmware applica una doppia protezione hardware-software:

- `dt <= 0.001 s`: Evita eccezioni matematiche di divisione per zero in caso di loop eseguiti troppo rapidamente.
- `dt > 0.5 s`: Evita picchi brutali della componente derivativa in caso di latenze impreviste sul bus I2C, "clippando" il passo temporale a un massimo di mezzo secondo.

12.2.2. Parametri Operativi del Controllore

La seguente tabella riassume i guadagni di sintonizzazione (*Tuning Gains*) e i limiti meccanici operativi imposti per confinare l'inviluppo di volo.

Asse di Controllo	K_p **(Prop)**	K_i **(Int)**	K_d **(Deriv)**
Altitudine (`_alt`)	0.5	0.05	0.2
Beccheggio (`_pitch`)	1.2	0.05	0.5
Rollio (`_roll`)	1.2	0.05	0.5
Autothrottle (`_vel`)	1.5	0.1	0.5

Costante di Vincolo	**Valore**	**Descrizione**
`ALTEZZA_MAX / MIN`	120 m/10 m	Quota di inviluppo (*Hard Deck* e *Ceiling*)
`MAX_PITCH / ROLL`	20°/35°	Limite aerodinamico strutturale
`GAS_MINIMO / MASSIMO`	25/130	Valori operativi di segnale PWM motore

12.2.3. Loop 1: Altitudine e Pitch-Targeting

L'anello di controllo altimetrico è il loop più esterno (*Outer Loop*) della dinamica longitudinale. L'errore viene calcolato come scostamento tra l'altitudine geodetica desiderata e l'AGL attuale ricavato dal barometro. Invece di comandare direttamente le superfici di volo, l'uscita di questo PID genera un angolo di beccheggio bersaglio (`targetPitch_Auto`).

Prima dell'elaborazione matematica, il sistema esegue un controllo rigoroso sui limiti dell'infrastruttura aerea (*Hard Bounds*):

- Se Quota > 120m, il PID viene escluso. Viene forzato un beccheggio di picchiata di $-8°$ e la base del gas viene ridotta a `GAS_MINIMO + 5` per prevenire il superamento della VNE (*Velocity Never Exceed*).
- Se Quota < 10m, il velivolo forza un assetto di cabrata d'emergenza di $+12°$ e innalza il gas a `GAS_MASSIMO - 10` per evitare il CFIT (*Controlled Flight Into Terrain*).

L'integratore di altitudine è protetto da un vincolo di *Anti-Windup* confinato tra ± 20.0.

12.2.4. Loop 2 e 3: Dinamica d'Assetto (Pitch e Roll)

Gli anelli interni (*Inner Loops*) di beccheggio e rollio elaborano l'errore tra i target (richiesti rispettivamente dal PID di Altitudine e dall'algoritmo di Navigazione L1) e gli angoli reali istantanei forniti dall'IMU BNO055.

Sia $e(t)$ l'errore angolare al tempo t, l'equazione implementata in tempo discreto per ogni asse è:

$$Output(t) = K_p \cdot e(t) + K_i \sum_{0}^{t} e(t)\Delta t + K_d \frac{e(t) - e(t-1)}{\Delta t} \tag{7}$$

Anche in queste sezioni, gli integratori sono confinati all'interno di un intervallo di sicurezza (±40.0). L'output finale di questi algoritmi, destinato alla matrice di mixing, subisce un *clipping* finale per impedire al drone di comandare i servomotori oltre i limiti di progetto (`MAX_PITCH` e `MAX_ROLL`).

12.2.5. Loop 4: Autothrottle e Gestione dell'Energia Cinetica

La regolazione della trazione è sganciata dalla dinamica di assetto, realizzando un vero e proprio Autothrottle aeronautico. L'algoritmo confronta la `targetVelocita` (impostata dalla navigazione a 60 km/h in rotta e a 45 km/h in fase di avvicinamento al target) con la `velocitaAttuale`, frutto della *Sensor Fusion* tra GPS e Tubo di Pitot.

Il calcolo della spinta è strutturato secondo la logica *Feed-Forward* + PID. Alla base del calcolo non si parte da zero, ma da un valore di `gasCorrente`, dettato in tempo reale dall'anello dell'altitudine (che può tagliare o aumentare il gas in caso di picchiata/cabrata d'emergenza). A questo valore di base viene sommato algebricamente l'output correttivo del PID di velocità. Il risultato è infine confinato nei limiti costruttivi del controllore ESC (`GAS_MINIMO` ... `GAS_MASSIMO`) ed erogato all'attuatore.

13. Attuazione Fisica, Override di Sicurezza e Watchdog Hardware

A valle di tutti i calcoli cinematici e della risoluzione delle matrici PID, il ciclo di esecuzione del Flight Controller converge verso la fase di attuazione fisica (*Actuation Phase*). In questo blocco critico, il sistema funge da arbitro finale (*Multiplexer* di sicurezza), decidendo se instradare i comandi di volo verso le superfici aerodinamiche o se interdire completamente il velivolo per motivi di emergenza.

13.1. Logica di Attuazione e Inibizione Motore (Crash Override)

Prima di autorizzare la scrittura dei segnali PWM (Pulse Width Modulation) sui pin di uscita, il firmware esegue un controllo assoluto e bloccante sulla variabile di stato globale `statoSchiantoRilevato`.

Qualora l'Inertial Measurement Unit (IMU) abbia precedentemente registrato un picco anomalo di accelerazione (superiore alla soglia G di guardia), il sistema innesca un *Hardware Override*. In questa condizione, l'algoritmo ignora deliberatamente qualsiasi output calcolato dal PID o richiesto dal pilota, e inietta un valore nullo (`motore.write(0)`) all'Electronic Speed Controller (ESC). Questa inibizione totale

azzera la spinta, mettendo in sicurezza la zona d'impatto ed evitando l'autodistruzione del propulsore contro eventuali ostacoli a terra.

Viceversa, se l'aeromobile opera in condizioni nominali (nessuna collisione rilevata), il flusso logico entra nel ramo condizionale standard. Solo ed esclusivamente in questo caso vengono inviati i comandi di attuazione:

1. Viene richiamata la routine di interfaccia `gestisci_allarmi()` per aggiornare il pannello LED.
2. I valori angolari finali (`correzionePitch` e `correzioneRoll`) vengono passati alla matrice adattiva del Mixer, che provvede al posizionamento fisico dei servomotori sulle cerniere alari.
3. La variabile `comandoGasFinale` viene trasmessa all'ESC, modulando la trazione in base alle richieste dell'Autothrottle o dello stick manuale.

13.2. Gestione Dinamica dell'Interfaccia Visiva (HMI)

La funzione `gestisci_allarmi()` aggiorna in tempo reale la telemetria visiva *on-board*, utilizzando un'architettura a priorità discendente (*Cascading If-Else*) su tre diodi LED ad alta luminosità:

- **Priorità 0 (Allarme Critico Hardware):** Il sistema verifica i flag di diagnostica di corrente dei quattro servomotori (`estSX_Ok`, ecc.). La defezione di un singolo attuatore satura l'interfaccia: i LED Rosso, Verde e Blu vengono accesi simultaneamente a livello `HIGH`. L'istruzione `return` blocca l'esecuzione della funzione, congelando le luci in questo stato per segnalare l'avaria catastrofica.
- **Priorità 1 (Allarme Logico/Energetico):** Se l'hardware attuativo è intatto, viene valutata la condizione della telemetria radio (`failsafe`) e del voltaggio propulsivo (`batteriaBassa_motore`). Il verificarsi di una di queste criticità accende il LED Rosso, avvisando l'operatore di un degrado operativo imminente.
- **Priorità 2 e 3 (Stati Nominali):** Il LED Verde viene pilotato direttamente dal parsing NMEA del modulo GNSS: si illumina solo se l'oggetto `gps.location.isValid()` restituisce `true`, confermando il *Fix* 3D spaziale. Infine, il LED Blu segnala la transizione modale: si accende se l'operatore ingaggia l'algoritmo di navigazione L1 e il PID (Modalità 2), spegnendosi in caso di volo totalmente manuale.

14. Allocazione dei Comandi e Matrice di *Mixing* Adattiva

Il blocco funzionale preposto all'attuazione fisica della traiettoria è il modulo di *Control Allocation*, comunemente definito *Mixer*. Questa routine riceve in ingresso gli sforzi di controllo calcolati dai loop PID (o i segnali grezzi in caso di volo manuale), espressi come scostamenti angolari desiderati per il beccheggio (δ_{pitch}) e per il rollio (δ_{roll}). Il suo compito primario è tradurre tali richieste in angoli fisici assoluti per i quattro servomotori disposti lungo l'apertura alare.

Ciò che distingue questa architettura da un sistema aeromodellistico tradizionale è la sua natura intrinsecamente adattiva (*Fault-Tolerant Control Allocation*). Il mixer ricalcola dinamicamente la propria matrice di trasformazione in tempo reale, basandosi sui flag di diagnostica hardware e sullo stato energetico del velivolo.

14.1. Risparmio Energetico Attivo (*Load Shedding*)

Nella sua fase di inizializzazione, l'algoritmo valuta lo stato di carica della batteria di servizio dell'avionica (`batteriaBassa_teensy`). Qualora il sistema si trovi in uno stato di *Failover* energetico (ovvero stia prelevando energia dal circuito di emergenza del motore), il firmware forza una procedura di distacco selettivo dei carichi, o *Load Shedding*. Le variabili di stato dei servomotori interni (`intSX_Ok` e `intDX_Ok`) vengono intenzionalmente invalidate. Questa logica degrada volontariamente la manovrabilità del velivolo per minimizzare l'assorbimento di corrente, preservando i milliampere residui per il sostentamento del microcontrollore, del GPS e del ricevitore radio.

14.2. Matrice Cinematica e Riconfigurazione Strutturale

Il nucleo matematico del modulo analizza le combinazioni booleane della salute degli attuatori (interni ed esterni) ed elabora quattro scenari operativi distinti, modificando l'equazione di governo delle superfici:

- **Caso A (Volo Nominale):** Se tutti i sensori INA219 confermano correnti operative standard, il velivolo sfrutta l'intera aerodinamica. I flap interni vengono dedicati esclusivamente al controllo del beccheggio (funzione di elevatore puro), mentre gli alettoni esterni gestiscono esclusivamente il rollio. Definito $\theta_{centro} = 90°$, la cinematica è:

$$\theta_{int_SX/DX} = \theta_{centro} + \delta_{pitch} \tag{8}$$

$$\theta_{ext_SX/DX} = \theta_{centro} \pm \delta_{roll} \tag{9}$$

- **Caso B (Avaria Superfici Interne):** Se le superfici di coda o i flap interni subiscono un guasto (o vengono disabilitati dal *Load Shedding*), il sistema

transiziona istantaneamente in configurazione *Elevon* (Elevator + Aileron). Le superfici esterne si fanno carico di entrambi gli assi di rotazione:

$$\theta_{ext_SX} = \theta_{centro} + \delta_{pitch} + \delta_{roll} \tag{10}$$

$$\theta_{ext_DX} = \theta_{centro} + \delta_{pitch} - \delta_{roll} \tag{11}$$

- **Caso C (Avaria Superfici Esterne):** Simmetricamente al Caso B, la perdita degli alettoni esterni comporta il trasferimento della matrice di *Elevon mixing* sui servomotori interni, permettendo al drone di mantenere la governabilità completa, seppur con un rateo di rollio ridotto a causa del minore braccio di leva aerodinamico.
- **Caso D (Avaria Totale):** L'assenza simultanea di attuatori validi su entrambi i segmenti alari causa l'interruzione della routine per impossibilità meccanica di manovra, generando un log di errore critico.

14.3. Gestione Dinamica dell'Hardware (Attach/Detach)

A valle del calcolo matematico, il firmware implementa un controllo logico sulle istanze degli oggetti `Servo`. Attraverso il monitoraggio dello stato precedente, il sistema invoca i metodi `attach()` e `detach()` solo durante le transizioni di stato.

L'operazione di `detach()` rimuove fisicamente la generazione del segnale PWM sul pin del microcontrollore. Dal punto di vista ingegneristico, questo rende il servomotore guasto "libero" (flottante), azzerando la corrente di mantenimento (*Holding Current*) ed evitando che un attuatore con la cerniera bloccata possa assorbire correnti di corto circuito (fino a 2.5A), le quali causerebbero l'immediato *Brownout* dell'intero sistema.

14.4. Saturazione Meccanica e Attuazione Finale

Prima di procedere all'istruzione di scrittura sui registri hardware (`write()`), i valori angolari elaborati dalle matrici vengono fatti passare attraverso una funzione di vincolo matematico (`constrain`). Tutti gli angoli vengono rigidamente confinati nell'intervallo operativo di sicurezza $[45°, 135°]$. Questa tecnica di *Saturazione Meccanica Software* è indispensabile per prevenire il finecorsa fisico dei leveraggi aerodinamici: tentare di spingere un servomotore oltre i propri limiti strutturali comporterebbe un picco di assorbimento anomalo, che verrebbe immediatamente (ed erroneamente) interpretato dalla routine di diagnostica come un guasto, innescando un falso positivo.

15. Watchdog Seriale del Sottosistema GNSS

L'ultima istruzione del blocco funge da *Hardware Watchdog* (cane da guardia) per la periferica seriale di navigazione. Sfruttando il contatore di sistema `millis()`, il firmware attende una finestra temporale di *grace period* pari a 10.000 ms (10 secondi) dall'avvio del microcontrollore.

Scaduto questo termine, il codice ispeziona il buffer dell'interfaccia UART (Serial1). Se il parser della libreria TinyGPSPlus ha processato un numero di caratteri esiguo (inferiore a 10 byte), il sistema deduce matematicamente l'impossibilità di stabilire un *handshake* NMEA valido. Questa condizione non segnala la semplice mancanza di satelliti al chiuso, bensì un problema di livello fisico (Layer 1 del modello OSI): cablaggi TX/RX invertiti, cavi tranciati o *baud rate* incompatibile. L'evento genera un avviso diagnostico esplicito sul terminale seriale, guidando l'operatore nella risoluzione del guasto elettrico prima ancora del rullaggio.

15.1. Sistema di Telemetria

All'interno del ciclo principale di esecuzione, il firmware invoca periodicamente la routine `inviaTelemetria()`, responsabile della trasmissione dei dati di stato del velivolo verso la Ground Control Station (GCS) tramite collegamento radio (LoRa).

Per evitare la saturazione della banda trasmissiva e garantire stabilità del link, l'invio dei dati è limitato a una frequenza di 2 Hz (ovvero un pacchetto ogni 500 ms), implementata tramite confronto temporale basato sulla funzione `millis()`.

15.1.1. Acquisizione Dati

Ad ogni ciclo di trasmissione, il sistema effettua una raccolta completa delle grandezze di interesse:

- Tensioni di alimentazione tramite moduli INA219 (motore, logica e servi)
- Assetto del velivolo (pitch, roll, yaw) da IMU
- Altitudine relativa da sensore barometrico
- Velocità (Pitot, GPS e stimata)
- Parametri di navigazione (distanza e rotta verso il target)
- Input del radiocomando (canali grezzi)
- Output dei controllori PID
- Stato e posizione dei servomeccanismi
- Temperature di sistema

- Dati GPS (satelliti, latitudine, longitudine)

15.1.2. Codifica degli Allarmi

Le condizioni critiche del sistema vengono codificate in un intero (`codiceAllarme`) utilizzando una struttura a *bitmask*, che consente la rappresentazione compatta di più stati booleani:

Bit	Significato
0	Failsafe attivo
1	Batteria motore scarica
2	Relè di ridondanza attivato
3	Batteria logica (Teensy) scarica
4	Impatto rilevato
5	Drone in volo

15.1.3. Struttura del Pacchetto

I dati vengono trasmessi sotto forma di stringa seriale delimitata da virgole (*CSV-like*), preceduta da un carattere di start (`$`) per facilitarne il parsing lato ricevitore.

La struttura del pacchetto è la seguente:

Indice	Contenuto
0	Start ($)
1	Modalità di volo
2	Codice allarmi (bitmask)
3–8	Tensioni (motore, Teensy, servi)
9	Stato salute servi (bit concatenati)
10–12	Pitch, Roll, Yaw
13	Altitudine AGL
14–16	Velocità (Pitot, GPS, stimata)
17–19	Navigazione (distanza, rotta, roll target)
20–22	Input radiocomando
23–25	Output controllori
26–29	Posizione servomeccanismi
30–31	Temperature
32–34	GPS (satelliti, latitudine, longitudine)
35	Stato relè ridondanza

15.1.4. Esempio di Pacchetto Telemetrico

Si riporta di seguito un esempio reale di pacchetto telemetrico trasmesso dal sistema:

`$,2,4,14.80,5.02,5.10,5.08,5.12,5.09,1111,2.3,-1.5,178.2,12.5,45.2,43.8,44.5,120,185`

Analisi del pacchetto

- `$` : carattere di start, indica l'inizio del pacchetto
- `2` : modalità di volo (2 = automatica)
- `4` : codice allarme (bit 2 attivo → relè di ridondanza inserito)
- `14.80` : tensione batteria motore (V)
- `5.02` : tensione alimentazione Teensy (V)
- `5.10, 5.08, 5.12, 5.09` : tensioni linee servi (V)
- `1111` : stato salute servi (tutti funzionanti)
- `2.3` : pitch (gradi)
- `-1.5` : roll (gradi)
- `178.2` : yaw / heading (gradi)
- `12.5` : altitudine relativa AGL (m)
- `45.2` : velocità aria (km/h, Pitot)
- `43.8` : velocità GPS (km/h)
- `44.5` : velocità stimata fusa (km/h)
- `120` : distanza dal target (m)
- `185.4` : rotta verso target (gradi)
- `10.2` : roll target (output navigazione L1)
- `1500, 1498, 1200` : input radiocomando (pitch, roll, gas)
- `10, -5, 1300` : output controllori (PID pitch, roll, gas)
- `90, 92, 88, 91` : posizione attuale servi (gradi)
- `45.6` : temperatura motore (°C)
- `22.3` : temperatura avionica (°C)
- `12` : numero satelliti GPS
- `41.902782` : latitudine
- `12.496366` : longitudine

- `1` : stato relè (1 = attivo, alimentazione su BEC)

Osservazioni Il pacchetto presenta una struttura deterministica a lunghezza variabile ma con ordine fisso dei campi, che consente un parsing robusto lato ricevitore. L'utilizzo di valori numerici in formato ASCII separati da virgole rappresenta un compromesso tra leggibilità, semplicità implementativa ed efficienza trasmissiva.

L'esempio mostrato evidenzia una condizione operativa nominale con sistema in volo automatico, tutti i servi funzionanti e relè di ridondanza attivo, indicativo di una precedente condizione di sottotensione sulla linea primaria.

15.1.5. Formato e Robustezza

Ogni valore numerico viene trasmesso con una precisione controllata (tipicamente 1–2 cifre decimali) per ridurre il payload e migliorare l'efficienza trasmissiva.

In assenza di segnale GPS valido, il sistema trasmette valori nulli predefiniti (`0, 0.000000, 0.000000`) per mantenere invariata la struttura del pacchetto e garantire la robustezza del parsing lato GCS.

15.1.6. Considerazioni Sistemistiche

L'approccio adottato presenta i seguenti vantaggi:

- Basso overhead computazionale e semplicità di parsing
- Elevata compatibilità con sistemi di monitoraggio e logging
- Robustezza agli errori grazie alla struttura fissa del pacchetto
- Scalabilità per l'aggiunta di nuovi parametri

La telemetria rappresenta quindi un elemento fondamentale per il monitoraggio in tempo reale, la diagnostica e la sicurezza operativa del sistema UAV.

Modulo / Sensore / Costante	**Collegamento / Significato**
TinyGPSPlus gps	Collegato a `Serial1` (GPS_SERIAL), baud 9600
Adafruit_BNO055 giroscopio	I2C default, indirizzo 0x28
Adafruit_BMP3XX barometro	I2C default
PIN_T_motore	A1
Global_Temperatura_motore	variabile per la lettura temperatura motore
temp_aria_barometro	variabile per temperatura barometro
TELEMETRIA	Serial4
SBUS ricevente	Serial3
canaliRC[16]	array per canali ricevuti dalla SBUS
PIN_ARIA	A0 (Pitot)
DENSITA_ARIA	1.225 kg/m^3
FATTORE_CONVERSIONE_PA	3.22
pinIntSX	6 (servo flap interno SX)
pinIntDX	7 (servo flap interno DX)
pinEstSX	8 (servo flap esterno SX)
pinEstDX	9 (servo flap esterno DX)
pinMotore	10 (servo motore)
servoInternoSX	Flap interno sinistro (pitch)
servoInternoDX	Flap interno destro (pitch)
servoEsternoSX	Flap esterno sinistro (pitch + roll)
servoEsternoDX	Flap esterno destro (pitch + roll)
motore	Servo motore principale
sensoreMotore	INA219 0x40
sensoreIntSX	INA219 0x41
sensoreIntDX	INA219 0x42
sensoreEstSX	INA219 0x43
sensoreEstDX	INA219 0x43, bus Wire1
sensoreTeensy	INA219 0x40, bus Wire1
VALORE_BATT_MOTORE_BASSA	11.8 V
VALORE_BATT_TEENSY_BASSA	4.9 V
SOGLIA_G_SCHIANTO	25 G
CENTRO_SERVO	90°
MAX_ROLL	35°
MAX_PITCH	20°
ALTEZZA_MAX	120 m
ALTEZZA_MIN	10 m
GAS_MINIMO	25
GAS_CROCIERA	70
GAS_MASSIMO	130
VELOCITA_CROCIERA	60 m/s
VELOCITA_AVVICINAMENTO	45 m/s
DISTANZA_FRENATA	150 m
TARGET_LAT	41.902782° es
TARGET_LON	12.496366° es
ALTITUDINE_TARGET	40 m
PIN_LED_ROSSO_ALARM	2
PIN_LED_VERDE_GPS	3
PIN_LED_BLU_PID	4
PIN_BUZZER	5
PIN_RELE	20

Tabella 2: Modulo / sensore / costante e collegamento / significato